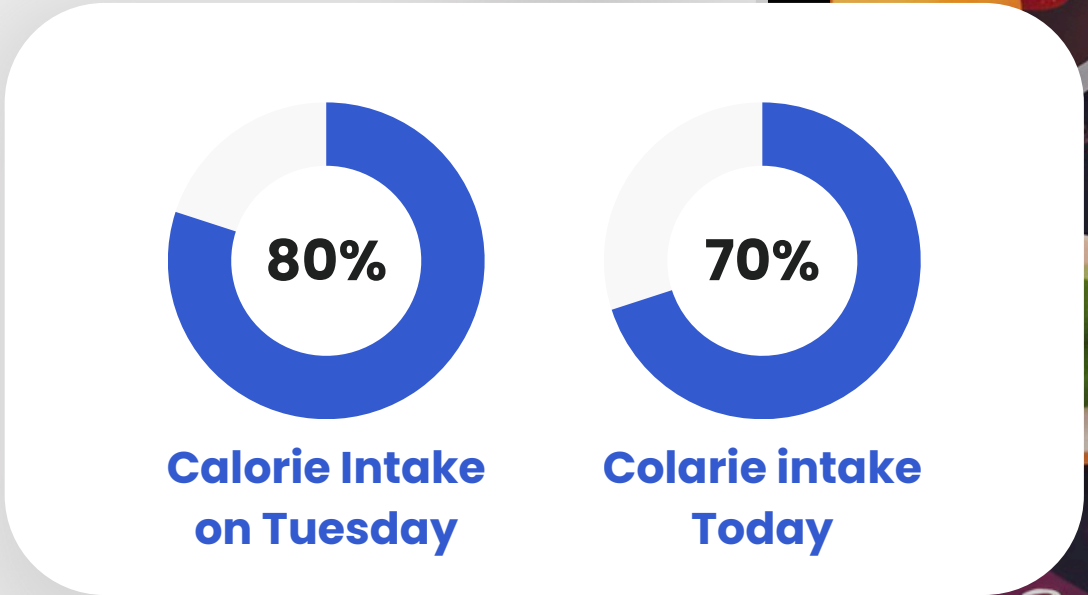
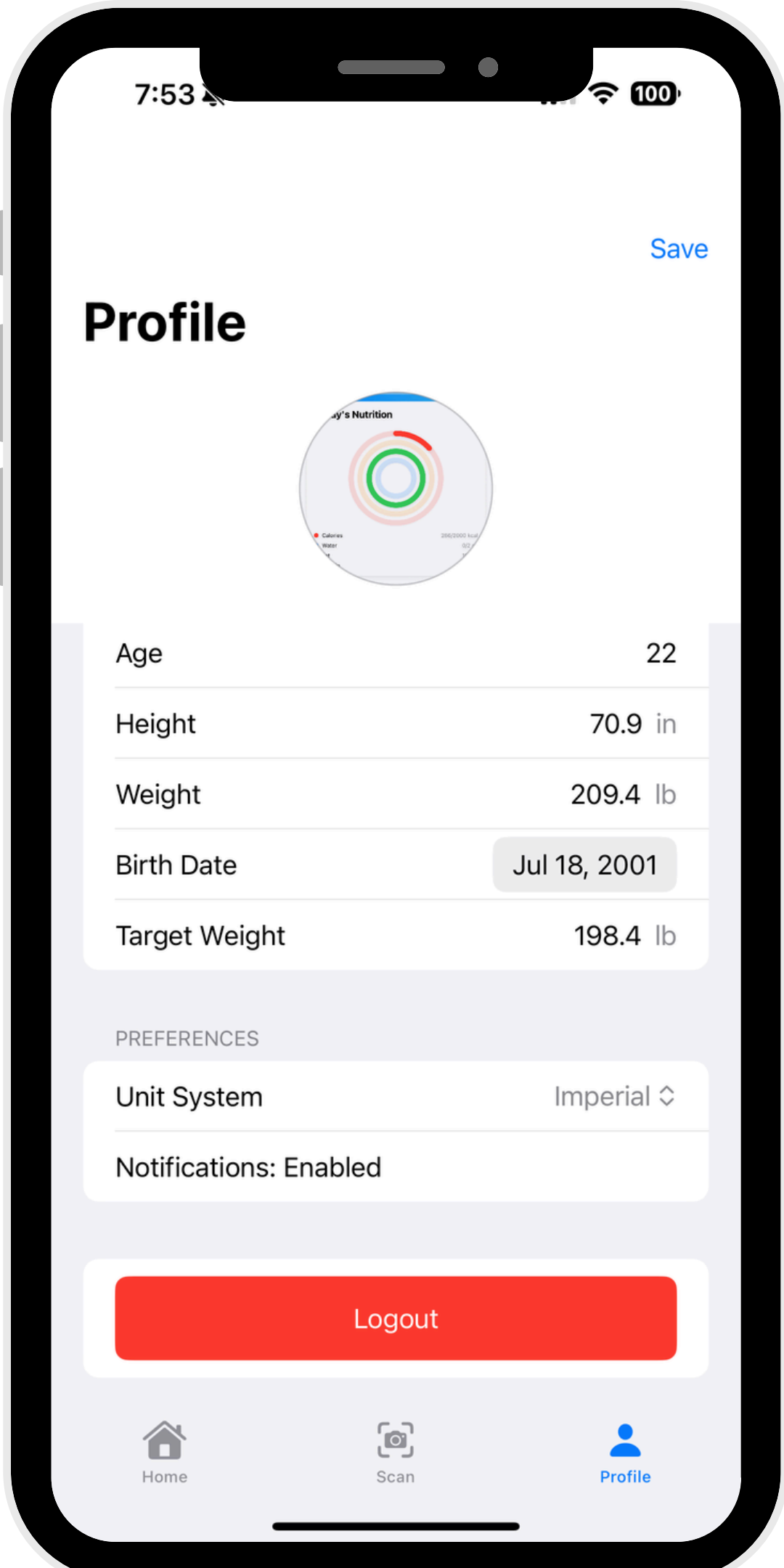


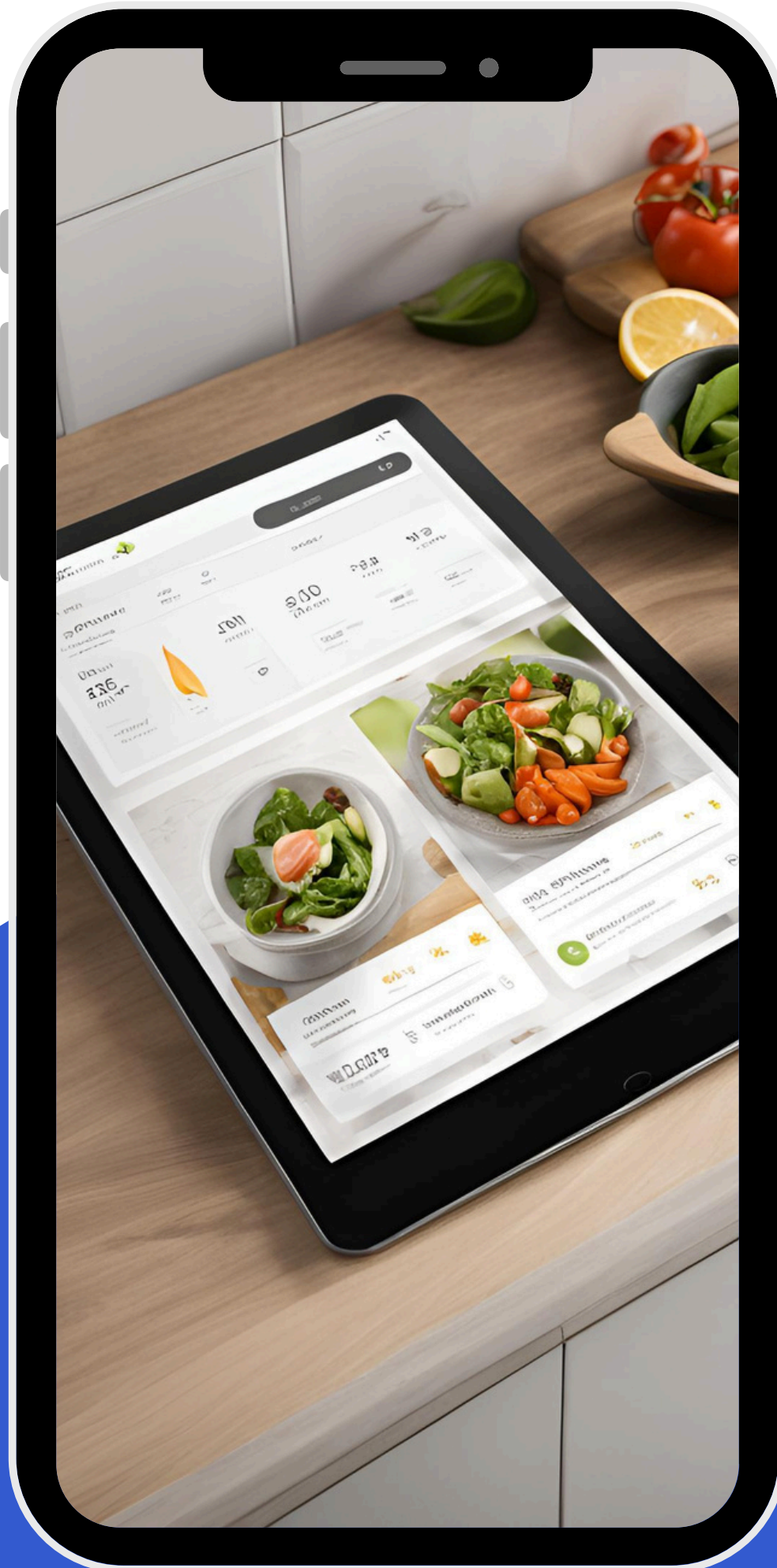
CalorieTrack: AI-Powered Nutrition for a Healthier You



Author Profile

Xiaohan Jiang: M.S. Computer Science, concentration on Data Science & Machine Learning. Equipped with python programming, Deep Learning Model Training and Analysis skills





Welcome To Our Calorie Track APP

CalorieTrack is an AI-powered mobile application designed to simplify calorie tracking and help users achieve their health goals. The app provides a personalized experience by calculating daily calorie allowances based on user inputs and leveraging advanced AI for meal recognition via image capture.

Calorie Track App Goals

Must Have

01 User onboarding and calorie goal calculation.



Have created the Calorie goal

02 AI-based meal recognition and calorie tracking.

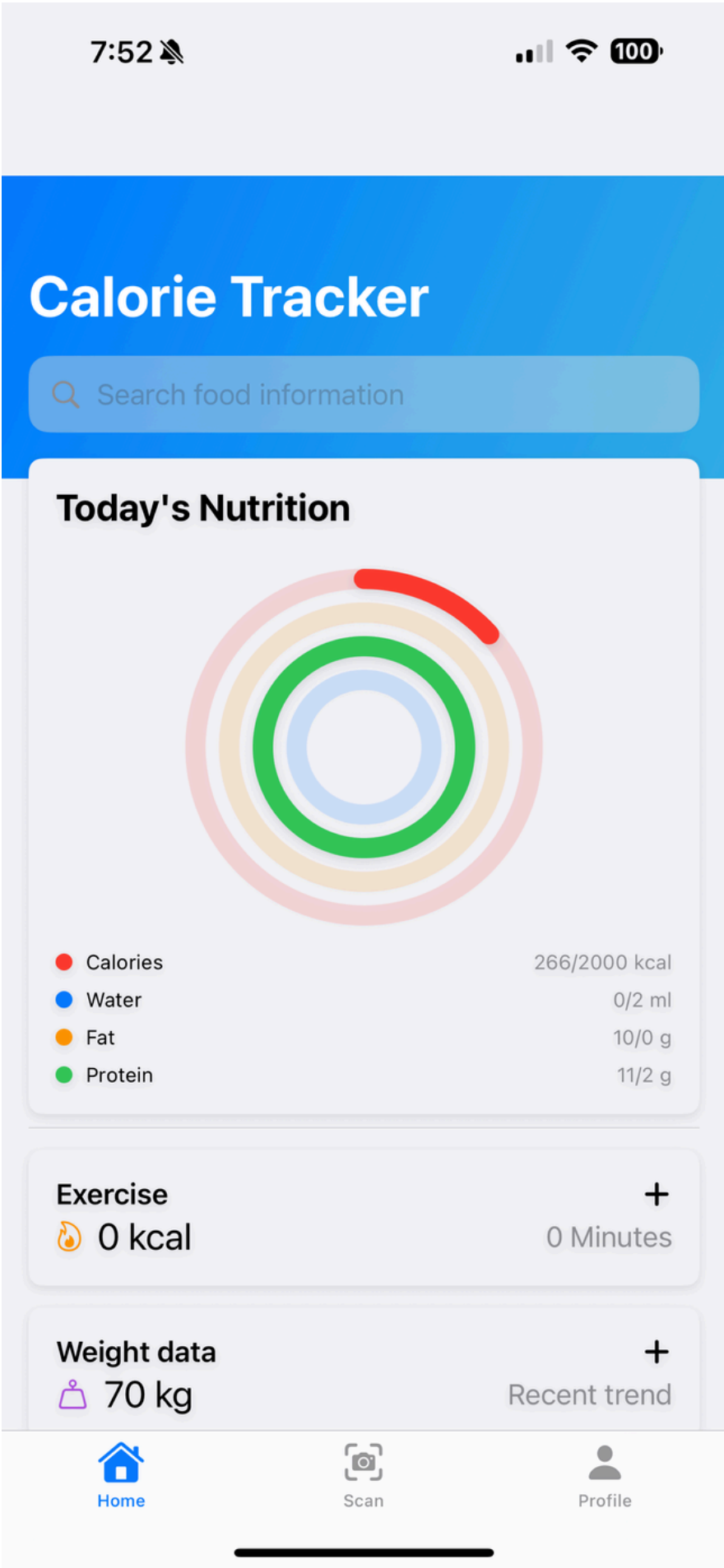


Trained the model

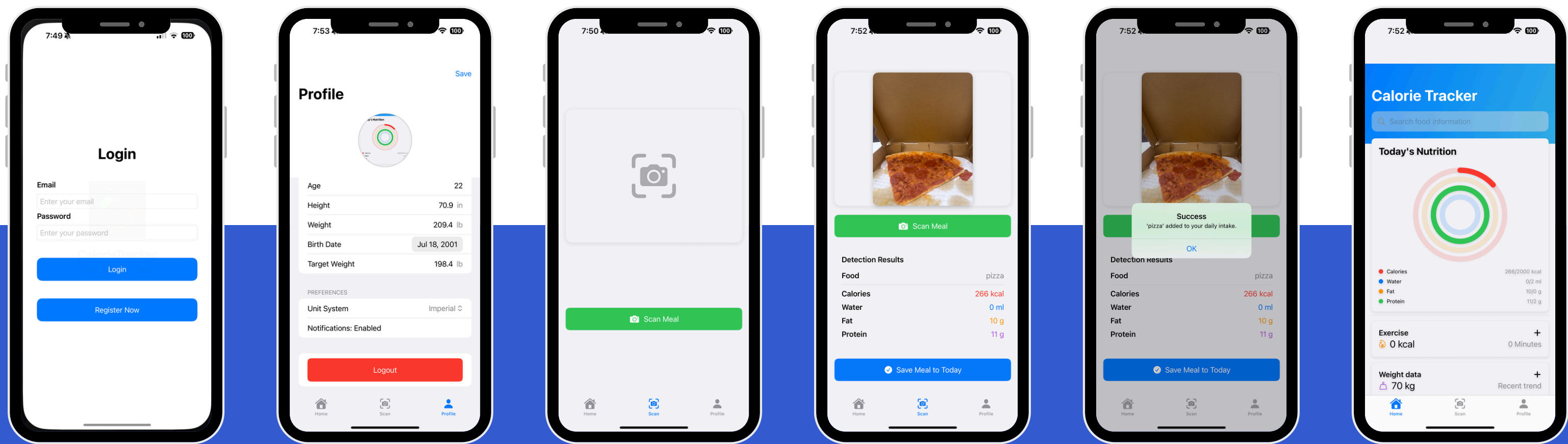
03 Real-time dashboard to display consumed and remaining calories.



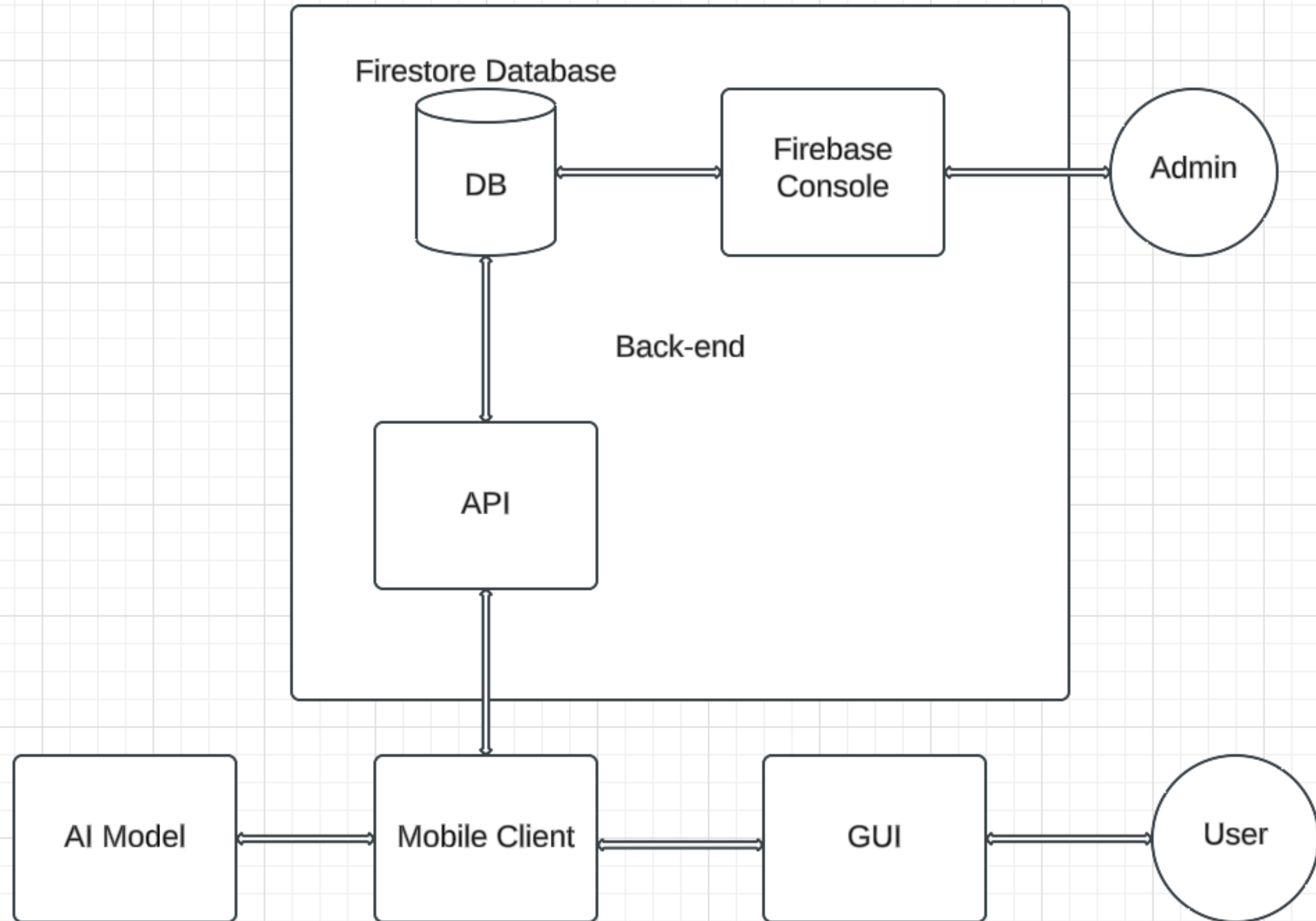
Real-time dashboard was created



Updated UI Screens



App Architecture



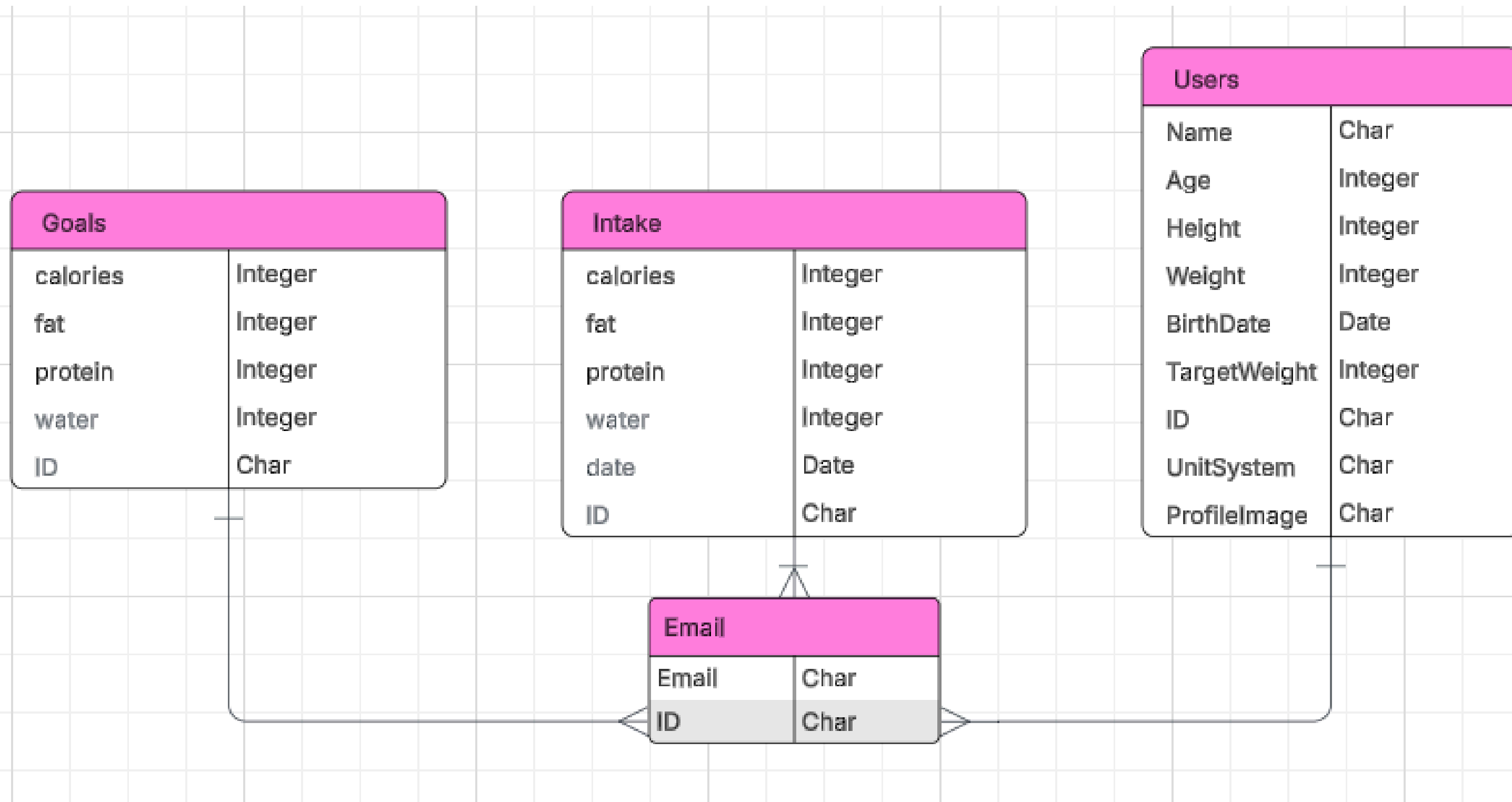
GUI is implemented by SWIFTUI.

Connection to Firebase is accomplished by FirebaseAuth and Firestore for the login function and send and get data from Firestore database.

In Mobile Client, camera function is fulfilled by AVFoundation(full featured framework for working with time-based audiovisual media) and Vision(Apply computer vision algorithms to perform a variety of tasks on input images and videos.)

CoreML is used to integrate our machine learning model to IOS app. Our app uses CoreML APIs and input images to make predictions of food.

Database Architecture



Model Implementation

1

Model Training

The CNN AI model is trained to recognize 101 categories of food and estimate portion sizes and calorie values.

2

Model Conversion

The .pth model is converted to Core ML format (.mlmodel) for compatibility with on-device inference.

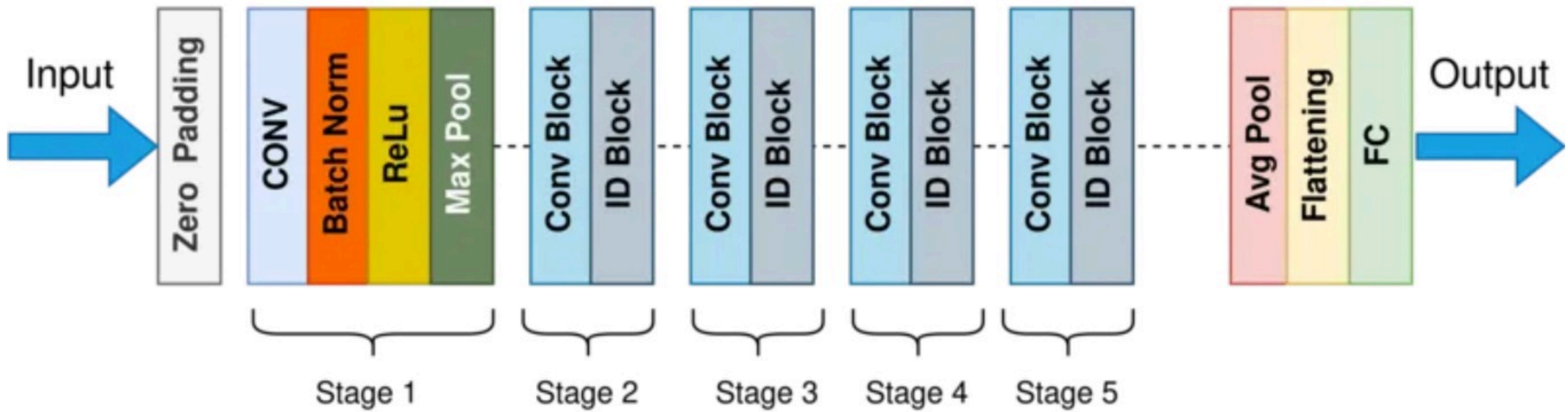
3

Model Integration

The AI model is integrated with the SwiftUI-based iOS app for smooth real-time predictions.

AI Model - CNN

ResNet50 Model Architecture



ResNet50 Architecture

AI Model - CNN

```
from google.colab import files
files.upload() # Upload kaggle.json when prompted
```

```
!kaggle datasets download -d kmader/food41
```

```
!unzip food41.zip -d food_data
!mv food_data/images food_data/food_images
```

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
from torchvision import datasets, models, transforms
from torch.utils.data import DataLoader
import numpy as np
import matplotlib.pyplot as plt
import os
import shutil
import random
from PIL import Image
import warnings
```

```
warnings.filterwarnings('ignore')
```

```
# Check for GPU and set device
device = torch.device("cuda:0" if torch.cuda.is_available()
else "cpu")
print(f"Using device: {device}")
```

```
# Data preparation (same as original)
base_dir = '/content/food_data/food_images'
train_dir = '/content/food_data/train'
val_dir = '/content/food_data/validation'
```

```
os.makedirs(train_dir, exist_ok=True)
os.makedirs(val_dir, exist_ok=True)

for category in os.listdir(base_dir):
    category_path = os.path.join(base_dir, category)
    if os.path.isdir(category_path):
        os.makedirs(os.path.join(train_dir, category),
exist_ok=True)
        os.makedirs(os.path.join(val_dir, category), exist_ok=True)
```

```
images = os.listdir(category_path)
random.shuffle(images)
split_index = int(0.8 * len(images))
```

```
for img in images[:split_index]:
    shutil.move(os.path.join(category_path, img),
os.path.join(train_dir, category))
for img in images[split_index:]:
    shutil.move(os.path.join(category_path, img),
os.path.join(val_dir, category))
```

```
# Data augmentation and normalization
train_transforms = transforms.Compose([
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(20),
    transforms.ColorJitter(brightness=0.2, contrast=0.2,
saturation=0.2, hue=0.1),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
0.225])
])
```

```
val_transforms = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
0.225])
])
```

```
# Create datasets and dataloaders
train_dataset = datasets.ImageFolder(train_dir,
train_transforms)
val_dataset = datasets.ImageFolder(val_dir, val_transforms,
```

```
class_names = train_dataset.classes
```

```
train_loader = DataLoader(train_dataset, batch_size=32,
shuffle=True, num_workers=4)
val_loader = DataLoader(val_dataset, batch_size=32,
shuffle=False, num_workers=4)
```

AI Model - CNN

```
# Display random food images
```

```
def imshow(inp, title=None):
```

```
    inp = inp.numpy().transpose((1, 2, 0))
```

```
    mean = np.array([0.485, 0.456, 0.406])
```

```
    std = np.array([0.229, 0.224, 0.225])
```

```
    inp = std * inp + mean
```

```
    inp = np.clip(inp, 0, 1)
```

```
    plt.imshow(inp)
```

```
    if title is not None:
```

```
        plt.title(title)
```

```
    plt.axis('off')
```

```
inputs, classes = next(iter(train_loader))
```

```
out = torchvision.utils.make_grid(inputs)
```

```
plt.figure(figsize=(10, 10))
```

```
imshow(out, title=[class_names[x] for x in classes])
```

```
plt.show()
```

```
# Create model
```

```
model = models.resnet50(pretrained=True)
```

```
# Freeze base layers
```

```
for param in model.parameters():
```

```
    param.requires_grad = False
```

```
# Modify classifier
```

```
num_ftrs = model.fc.in_features
```

```
model.fc = nn.Sequential(
```

```
    nn.Linear(num_ftrs, 1024),
```

```
    nn.ReLU(),
```

```
    nn.Linear(1024, len(class_names))
```

```
)
```

```
model = model.to(device)
```

```
# Loss and optimizer
```

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.Adam(model.fc.parameters())
```

```
# Training loop
```

```
num_epochs = 6
```

```
best_acc = 0.0
```

```
train_losses, val_losses = [], []
```

```
train_accs, val_accs = [], []
```

```
for epoch in range(num_epochs):
```

```
    print(f'Epoch {epoch+1}/{num_epochs}')
```

```
    print('-' * 10)
```

```
#Training phase
```

```
model.train()
```

```
running_loss = 0.0
```

```
running_corrects = 0
```

```
for inputs, labels in train_loader:
```

```
    inputs = inputs.to(device)
```

```
    labels = labels.to(device)
```

```
    optimizer.zero_grad()
```

```
    outputs = model(inputs)
```

```
    loss = criterion(outputs, labels)
```

```
    _, preds = torch.max(outputs, 1)
```

```
    loss.backward()
```

```
    optimizer.step()
```

```
    running_loss += loss.item() * inputs.size(0)
```

```
    running_corrects += torch.sum(preds == labels.data)
```

```
epoch_loss = running_loss / len(train_dataset)
```

```
epoch_acc = running_corrects.double() / len(train_datase
```

```
train_losses.append(epoch_loss)
```

```
train_accs.append(epoch_acc)
```

```
print(f'Train Loss: {epoch_loss:.4f} Acc: {epoch_acc:.4f}')
```

AI Model - CNN

```
# Validation phase
model.eval()
val_running_loss = 0.0
val_running_corrects = 0

with torch.no_grad():
    for inputs, labels in val_loader:
        inputs = inputs.to(device)
        labels = labels.to(device)

        outputs = model(inputs)
        loss = criterion(outputs, labels)
        _, preds = torch.max(outputs, 1)

        val_running_loss += loss.item() * inputs.size(0)
        val_running_corrects += torch.sum(preds == labels.data)

    val_epoch_loss = val_running_loss / len(val_dataset)
    val_epoch_acc = val_running_corrects.double() / len(val_dataset)

    val_losses.append(val_epoch_loss)
    val_accs.append(val_epoch_acc)

print(f'Val Loss: {val_epoch_loss:.4f} Acc: {val_epoch_acc:.4f}\n')

# Save best model
if val_epoch_acc > best_acc:
    best_acc = val_epoch_acc
    torch.save(model.state_dict(), 'best_model.pth')
```

```
# Load best model weights
model.load_state_dict(torch.load('best_model.pth'))

# Save model
torch.save(model.state_dict(), 'food_classification_model.pth')

# Inference function
def predict_image(image_path):
    img = Image.open(image_path)
    img = val_transforms(img).unsqueeze(0).to(device)

    model.eval()
    with torch.no_grad():
        output = model(img)
        _, predicted = torch.max(output, 1)

    class_name = class_names[predicted.item()]

    plt.imshow(Image.open(image_path))
    plt.title(f'Predicted: {class_name}')
    plt.axis('off')
    plt.show()

# Upload and predict
from google.colab import files

uploaded = files.upload()
for filename in uploaded.keys():
    predict_image(filename)
```


Mobile App – DEMO

IOS Application

Risks for the IOS application:

1. Bias or Inaccuracy in AI Embedding Representations.

Solution: Ensure that the AI model is trained on a wide and diverse dataset that includes various cuisines.

2. Lots of traffic may slow down the database, leading to performance issues. Unused or duplicate accounts may clutter storage.

Solution: Implement indexing, optimize queries, and periodically clean inactive accounts to maintain efficiency.

Concerns for the IOS application:

1. The lack of training dataset will cause the insufficiency of the image prediction.
2. Limitation of input length of Firestore database, cannot save a large image using base64 encode.

Challenges while connecting the three parts:

1. Trained the AI model using the PyTorch instead of TensorFlow in order to save the pre-trained model as a CoreML file to connect to IOS app.
2. The limited CUDA space limits me to train more epochs on the model.

Further improvements for the IOS application:

1. AI model monthly maintenance to improve the predicting result accuracy.
2. Add in exercise record function.

Thank you for watching



CalorieTracker

Track. Achieve. Thrive.